

Phần 3: Ánh xạ Mối quan hệ trong JPA

3.1 One-to-One Relationship

Mối quan hệ một-một giữa hai Entity, trong đó mỗi instance của Entity A tương ứng với tối đa một instance của Entity B và ngược lại.

3.1.1 Unidirectional One-to-One

Trong mối quan hệ một-một đơn hướng, chỉ có một Entity biết về sự tồn tại của Entity kia. Ví dụ, `User` biết về `UserProfile`, nhưng `UserProfile` không biết về `User`.

Java

```
@Entity
public class User {
    @Id
    @GeneratedValue
    private Long id;
    private String name;

    @OneToOne(cascade = CascadeType.ALL) // Định nghĩa mối quan hệ một-một.
    // CascadeType.ALL nghĩa là mọi thao tác (persist, merge, remove,...) trên User
    // cũng sẽ được áp dụng cho UserProfile liên quan.
    @JoinColumn(name = "profile_id") // Chỉ định cột khóa ngoại (foreign key)
    // trong bảng User sẽ tham chiếu đến khóa chính của bảng UserProfile. Tên cột sẽ
    // là profile_id.
    private UserProfile profile;
    // getters, setters
}

@Entity
public class UserProfile {
    @Id
    @GeneratedValue
    private Long id;
    private String bio;
    private String website;
    // getters, setters
}
```

Giải thích:

- `@OneToOne` : Annotation này được sử dụng để thiết lập mối quan hệ một-một giữa hai Entity.
- `cascade = CascadeType.ALL` : Khi bạn thực hiện một thao tác (ví dụ: lưu, xóa) trên Entity `User`, thao tác tương tự cũng sẽ được áp dụng cho Entity `UserProfile` được liên kết. Điều này giúp quản lý vòng đời của các Entity liên quan một cách tự động.
- `@JoinColumn(name = "profile_id")` : Annotation này chỉ định cột khóa ngoại trong bảng `User` sẽ được sử dụng để liên kết với bảng `UserProfile`. Cột `profile_id` sẽ chứa ID của `UserProfile` tương ứng.

3.1.2 Bidirectional One-to-One

Trong mối quan hệ một-một hai chiều, cả hai Entity đều biết về sự tồn tại của nhau. Ví dụ, `User` biết về `UserProfile` và `UserProfile` cũng biết về `User`.

Java

```
@Entity
public class User {
    @Id
    @GeneratedValue
    private Long id;
    private String name;

    @OneToOne(cascade = CascadeType.ALL) // Phía sở hữu mối quan hệ. Cột khóa
    ngoại sẽ nằm trong bảng User.
    @JoinColumn(name = "profile_id") // Chỉ định cột khóa ngoại trong bảng
    User.
    private UserProfile profile;
    // getters, setters
}

@Entity
public class UserProfile {
    @Id
    @GeneratedValue
    private Long id;
    private String bio;
    private String website;

    @OneToOne(mappedBy = "profile") // Phía không sở hữu mối quan hệ.
    mappedBy chỉ ra trường nào trong Entity User đang sở hữu mối quan hệ này.
    private User user;
```

```
// getters, setters  
}
```

Giải thích:

- Trong mỗi quan hệ hai chiều, một bên được coi là "owning side" (phía sở hữu) và bên kia là "inverse side" (phía không sở hữu).
- `@JoinColumn` luôn được đặt ở phía sở hữu mối quan hệ (nơi cột khóa ngoại thực sự tồn tại trong cơ sở dữ liệu). Trong ví dụ này, `User` là phía sở hữu vì cột `profile_id` nằm trong bảng `User`.
- `mappedBy = "profile"` : Ở phía không sở hữu (`UserProfile`), chúng ta sử dụng `mappedBy` để chỉ ra rằng mối quan hệ này đã được ánh xạ bởi trường `profile` trong lớp `User`. Điều này báo cho JPA biết rằng `UserProfile` không tạo ra cột khóa ngoại riêng mà sử dụng cột đã được định nghĩa ở phía `User`.

3.1.3 Shared Primary Key

Trong mỗi quan hệ một-một sử dụng Shared Primary Key, khóa chính của một Entity cũng là khóa ngoại tham chiếu đến khóa chính của Entity kia. Điều này thường được sử dụng khi hai Entity có mối quan hệ phụ thuộc chặt chẽ và chia sẻ cùng một ID.

Java

```
@Entity  
public class User {  
    @Id  
    @GeneratedValue  
    private Long id;  
    private String name;  
  
    @OneToOne(mappedBy = "user", cascade = CascadeType.ALL) //  
    mappedBy="user" chỉ ra rằng trường user trong UserProfile là phía sở hữu.  
    private UserProfile profile;  
    // getters, setters  
}  
  
@Entity  
public class UserProfile {  
    @Id // Khóa chính của UserProfile  
    private Long id;
```

```

private String bio;

@OneToOne // Mỗi quan hệ một-một với User.
@MapsId // Chỉ định rằng khóa chính của UserProfile sẽ được ánh xạ từ
khóa chính của User.
@JoinColumn(name = "id") // Cột khóa ngoại có tên là id, tham chiếu đến
khóa chính của User.
private User user;
// getters, setters
}

```

Giải thích:

- Trong mô hình này, `UserProfile` không có khóa chính riêng biệt mà sử dụng khóa chính của `User` làm khóa chính và khóa ngoại của nó.
- `@MapsId` : Annotation này được sử dụng trên trường khóa chính của `UserProfile` để chỉ ra rằng giá trị của nó sẽ được lấy từ khóa chính của Entity được liên kết (`User`).
- `@JoinColumn(name = "id")` : Mặc dù tên cột là `id` , nhưng trong ngữ cảnh này, nó hoạt động như một khóa ngoại tham chiếu đến khóa chính của bảng `User` .

3.2 One-to-Many và Many-to-One Relationship

3.2.1 Unidirectional One-to-Many

Trong mối quan hệ một-nhiều đơn hướng, chỉ có Entity ở phía "một" biết về Entity ở phía "nhiều". Ví dụ, `Department` biết về `Employee` , nhưng `Employee` không biết về `Department` .

Java

```

@Entity
public class Department {
    @Id
    @GeneratedValue
    private Long id;
    private String name;

    @OneToMany(cascade = CascadeType.ALL, fetch = FetchType.LAZY) // Một
    Department có nhiều Employee. fetch = FetchType.LAZY nghĩa là danh sách
    employees sẽ chỉ được load khi cần thiết.
    @JoinColumn(name = "department_id") // Chỉ định cột khóa ngoại trong bảng
    Employee sẽ tham chiếu đến khóa chính của bảng Department. Tên cột sẽ là

```

```

department_id.
    private List<Employee> employees = new ArrayList<>();
    // getters, setters
}

@Entity
public class Employee {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    private String email;
    // getters, setters
}

```

Giải thích:

- `@OneToMany` : Annotation này được đặt trên trường `employees` trong lớp `Department` để chỉ ra rằng một `Department` có thể có nhiều `Employee` .
- `@JoinColumn(name = "department_id")` : Trong mối quan hệ một-nhiều đơn hướng, cột khóa ngoại (`department_id`) sẽ được tạo trong bảng của Entity "nhiều" (`Employee`). JPA sẽ quản lý cột này.
- `fetch = FetchType.LAZY` : Đây là chế độ tải mặc định cho `@OneToMany` và `@ManyToMany` . Dữ liệu của `employees` sẽ chỉ được tải từ cơ sở dữ liệu khi bạn thực sự truy cập vào danh sách này. Điều này giúp tối ưu hiệu suất bằng cách tránh tải quá nhiều dữ liệu không cần thiết.

3.2.2 Bidirectional One-to-Many/Many-to-One

Trong mối quan hệ một-nhiều hai chiều, cả hai Entity đều biết về nhau. `Department` biết về danh sách `Employee` của nó, và mỗi `Employee` biết về `Department` mà nó thuộc về.

Java

```

@Entity
public class Department {
    @Id
    @GeneratedValue
    private Long id;
    private String name;

```

```

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL, fetch =
FetchType.LAZY) // mappedBy="department" chỉ ra rằng trường department trong
Employee là phía sở hữu mối quan hệ này.
    private List<Employee> employees = new ArrayList<>();

    // Helper methods để đồng bộ hóa cả hai phía của mỗi quan hệ
    public void addEmployee(Employee employee) {
        employees.add(employee);
        employee.setDepartment(this); // Quan trọng: thiết lập mối quan hệ ở
phía Employee
    }

    public void removeEmployee(Employee employee) {
        employees.remove(employee);
        employee.setDepartment(null); // Quan trọng: gỡ bỏ mối quan hệ ở phía
Employee
    }
    // getters, setters
}

@Entity
public class Employee {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    private String email;

    @ManyToOne(fetch = FetchType.LAZY) // Một Employee thuộc về một
Department. fetch = FetchType.LAZY là mặc định cho ManyToOne.
    @JoinColumn(name = "department_id") // Chỉ định cột khóa ngoại trong bảng
Employee sẽ tham chiếu đến khóa chính của bảng Department.
    private Department department;
    // getters, setters
}

```

Giải thích:

- `@ManyToOne` : Luôn là phía sở hữu mối quan hệ trong mỗi quan hệ một-nhiều hai chiều. Cột khóa ngoại (`department_id`) sẽ được tạo trong bảng `Employee` .
- `@OneToMany(mappedBy = "department")` : Ở phía `Department` , chúng ta sử dụng `mappedBy` để chỉ ra rằng mối quan hệ này đã được ánh xạ bởi trường `department`

trong lớp `Employee` . Điều này báo cho JPA biết rằng `Department` không tạo ra cột khóa ngoại riêng mà sử dụng cột đã được định nghĩa ở phía `Employee` .

- **Helper methods (`addEmployee` , `removeEmployee`)**: Rất quan trọng để đảm bảo tính nhất quán của dữ liệu khi làm việc với mối quan hệ hai chiều. Khi bạn thêm hoặc xóa một `Employee` khỏi `Department` , bạn cần phải thiết lập hoặc gỡ bỏ mối quan hệ ở cả hai phía để tránh các vấn đề về dữ liệu không đồng bộ.

3.3 Many-to-Many Relationship

3.3.1 Unidirectional Many-to-Many

Trong mối quan hệ nhiều-nhiều đơn hướng, chỉ có một Entity biết về Entity kia. JPA sẽ tạo một bảng trung gian để quản lý mối quan hệ này.

Java

```
@Entity
public class Student {
    @Id
    @GeneratedValue
    private Long id;
    private String name;

    @ManyToMany(cascade = CascadeType.ALL) // Một Student có thể đăng ký
    // nhiều Course.
    @JoinTable( // Định nghĩa bảng trung gian để quản lý mối quan hệ nhiều-
    // nhiều.
        name = "student_course", // Tên của bảng trung gian.
        joinColumns = @JoinColumn(name = "student_id"), // Cột khóa ngoại
        // trong bảng trung gian tham chiếu đến khóa chính của Student.
        inverseJoinColumns = @JoinColumn(name = "course_id") // Cột khóa
        // ngoại trong bảng trung gian tham chiếu đến khóa chính của Course.
    )
    private Set<Course> courses = new HashSet<>();
    // getters, setters
}

@Entity
public class Course {
    @Id
    @GeneratedValue
    private Long id;
    private String title;
```

```
private String description;  
// getters, setters  
}
```

Giải thích:

- `@ManyToMany` : Annotation này được sử dụng để thiết lập mối quan hệ nhiều-nhiều.
- `@JoinTable` : Khi có mối quan hệ nhiều-nhiều, JPA cần một bảng trung gian để lưu trữ các liên kết giữa hai Entity. `@JoinTable` cho phép bạn tùy chỉnh tên của bảng trung gian và tên của các cột khóa ngoại trong bảng đó.
- `name` : Tên của bảng trung gian.
- `joinColumns` : Định nghĩa cột khóa ngoại trong bảng trung gian trỏ về Entity hiện tại (`Student`).
- `inverseJoinColumns` : Định nghĩa cột khóa ngoại trong bảng trung gian trỏ về Entity đối diện (`Course`).

3.3.2 Bidirectional Many-to-Many

Trong mối quan hệ nhiều-nhiều hai chiều, cả hai Entity đều biết về nhau. `Student` biết về các `Course` mà họ đăng ký, và `Course` biết về các `Student` đã đăng ký nó.

Java

```
@Entity  
public class Student {  
    @Id  
    @GeneratedValue  
    private Long id;  
    private String name;  
  
    @ManyToMany(cascade = CascadeType.ALL) // Phía sở hữu mối quan hệ. Bảng  
    // trung gian sẽ được định nghĩa ở đây.  
    @JoinTable(  
        name = "student_course",  
        joinColumns = @JoinColumn(name = "student_id"),  
        inverseJoinColumns = @JoinColumn(name = "course_id")  
    )  
    private Set<Course> courses = new HashSet<>();  
}
```

```

// Helper methods để đồng bộ hóa cả hai phía của mối quan hệ
public void addCourse(Course course) {
    courses.add(course);
    course.getStudents().add(this); // Quan trọng: thiết lập mối quan hệ
    ở phía Course
}

public void removeCourse(Course course) {
    courses.remove(course);
    course.getStudents().remove(this); // Quan trọng: gỡ bỏ mối quan hệ ở
    phía Course
}
// getters, setters
}

@Entity
public class Course {
    @Id
    @GeneratedValue
    private Long id;
    private String title;

    @ManyToMany(mappedBy = "courses") // Phía không sở hữu mối quan hệ.
    mappedBy chỉ ra trường nào trong Entity Student đang sở hữu mối quan hệ này.
    private Set<Student> students = new HashSet<>();
    // getters, setters
}

```

Giải thích:

- Trong mối quan hệ hai chiều, một bên được coi là "owning side" (phía sở hữu) và bên kia là "inverse side" (phía không sở hữu).
- `@JoinTable` luôn được đặt ở phía sở hữu mối quan hệ (`Student` trong ví dụ này) để định nghĩa bảng trung gian.
- `mappedBy = "courses"` : Ở phía không sở hữu (`Course`), chúng ta sử dụng `mappedBy` để chỉ ra rằng mối quan hệ này đã được ánh xạ bởi trường `courses` trong lớp `Student` . Điều này báo cho JPA biết rằng `Course` không tạo ra bảng trung gian riêng mà sử dụng bảng đã được định nghĩa ở phía `Student` .
- **Helper methods (`addCourse` , `removeCourse`)**: Tương tự như mối quan hệ One-to-Many hai chiều, việc sử dụng các helper method là rất quan trọng để đảm bảo tính nhất

quán của dữ liệu khi thêm hoặc xóa các liên kết trong mối quan hệ nhiều-nhiều hai chiều.

3.3.3 Many-to-Many với Entity trung gian

Khi bảng trung gian của mối quan hệ nhiều-nhiều có thêm các thuộc tính riêng (ví dụ: ngày đăng ký, điểm số), chúng ta không thể sử dụng `@ManyToMany` trực tiếp. Thay vào đó, chúng ta cần tạo một Entity riêng để đại diện cho bảng trung gian đó và ánh xạ mối quan hệ nhiều-nhiều thành hai mối quan hệ một-nhiều.

Java

```
@Entity
public class Student {
    @Id
    @GeneratedValue
    private Long id;
    private String name;

    @OneToMany(mappedBy = "student", cascade = CascadeType.ALL) // Một
    Student có nhiều Enrollment.
    private Set<Enrollment> enrollments = new HashSet<>();
    // getters, setters
}

@Entity
public class Course {
    @Id
    @GeneratedValue
    private Long id;
    private String title;

    @OneToMany(mappedBy = "course", cascade = CascadeType.ALL) // Một Course
    có nhiều Enrollment.
    private Set<Enrollment> enrollments = new HashSet<>();
    // getters, setters
}

@Entity
public class Enrollment {
    @Id
    @GeneratedValue
    private Long id;

    @ManyToOne // Một Enrollment thuộc về một Student.
```

```

    @JoinColumn(name = "student_id") // Cột khóa ngoại tham chiếu đến
    Student.
    private Student student;

    @ManyToOne // Một Enrollment thuộc về một Course.
    @JoinColumn(name = "course_id") // Cột khóa ngoại tham chiếu đến Course.
    private Course course;

    @Temporal(TemporalType.DATE) // Lưu trữ ngày đăng ký.
    private Date enrollmentDate;

    private Double grade; // Lưu trữ điểm số.
    // getters, setters
}

```

Giải thích:

- Thay vì sử dụng `@ManyToMany`, chúng ta tạo một Entity mới (`Enrollment`) để đại diện cho bảng trung gian. Entity này sẽ có khóa chính riêng và các thuộc tính bổ sung (`enrollmentDate`, `grade`).
- Mối quan hệ nhiều-nhiều giữa `Student` và `Course` được chuyển đổi thành hai mối quan hệ một-nhiều:
 - `Student` `OneToMany` `Enrollment`
 - `Course` `OneToMany` `Enrollment`
- Entity `Enrollment` sẽ có hai mối quan hệ `ManyToOne` với `Student` và `Course`, mỗi mối quan hệ sẽ có một cột khóa ngoại tương ứng (`student_id`, `course_id`).

Cách tiếp cận này cung cấp sự linh hoạt cao hơn khi bạn cần lưu trữ thông tin bổ sung về mối quan hệ giữa hai Entity.

3.4 Cascade Types

Cascade xác định các thao tác nào sẽ được truyền từ Entity cha sang Entity con. Điều này giúp tự động hóa việc quản lý vòng đời của các Entity liên quan.

Java

```

@Entity
public class Department {
    @OneToMany(mappedBy = "department", cascade = { // Định nghĩa các loại
        cascade.
            CascadeType.PERSIST, // Khi persist (lưu) Department, các Employee
            // liên quan cũng sẽ được persist.
            CascadeType.MERGE, // Khi merge (cập nhật) Department, các
            // Employee liên quan cũng sẽ được merge.
            CascadeType.REMOVE, // Khi remove (xóa) Department, các Employee
            // liên quan cũng sẽ bị xóa.
            CascadeType.REFRESH, // Khi refresh Department, các Employee liên
            // quan cũng sẽ được refresh (tải lại từ DB).
            CascadeType.DETACH // Khi detach Department, các Employee liên
            // quan cũng sẽ bị detach.
        })
    private List<Employee> employees;

    // Hoặc sử dụng CascadeType.ALL cho tất cả các loại cascade trên
    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
    private List<Employee> employees2;
}

```

Giải thích:

- `CascadeType.PERSIST` : Khi bạn lưu một Entity cha, tất cả các Entity con được liên kết cũng sẽ được lưu.
- `CascadeType.MERGE` : Khi bạn hợp nhất (merge) một Entity cha, tất cả các Entity con được liên kết cũng sẽ được hợp nhất.
- `CascadeType.REMOVE` : Khi bạn xóa một Entity cha, tất cả các Entity con được liên kết cũng sẽ bị xóa. Cần cẩn trọng khi sử dụng loại cascade này.
- `CascadeType.REFRESH` : Khi bạn làm mới (refresh) một Entity cha từ cơ sở dữ liệu, các Entity con được liên kết cũng sẽ được làm mới.
- `CascadeType.DETACH` : Khi bạn tách một Entity cha khỏi Persistence Context, các Entity con được liên kết cũng sẽ bị tách.
- `CascadeType.ALL` : Bao gồm tất cả các loại cascade trên. Thường được sử dụng để đơn giản hóa cấu hình, nhưng cần hiểu rõ tác động của nó.

3.5 Fetch Types

Xác định cách thức load dữ liệu liên quan (associated data) từ cơ sở dữ liệu. Có hai loại chính:

Java

```
@Entity
public class Department {
    // LAZY: Load khi cần thiết (default cho @OneToMany, @ManyToMany)
    @OneToMany(mappedBy = "department", fetch = FetchType.LAZY) // Dữ liệu
    của employees sẽ chỉ được tải khi bạn truy cập vào danh sách này.
    private List<Employee> employees;

    // EAGER: Load ngay lập tức (default cho @OneToOne, @ManyToOne)
    @OneToMany(mappedBy = "department", fetch = FetchType.EAGER) // Dữ liệu
    của employees sẽ được tải ngay lập tức cùng với Department.
    private List<Employee> employees2;
}
```

Giải thích:

- `FetchType.LAZY` (Tải lười): Dữ liệu của các Entity liên quan sẽ không được tải ngay lập tức khi Entity chính được tải. Thay vào đó, chúng sẽ chỉ được tải khi bạn thực sự truy cập vào chúng (ví dụ: gọi `getEmployees()`). Đây là chế độ tải mặc định cho các mối quan hệ `@OneToMany` và `@ManyToMany` và thường được khuyến nghị để tối ưu hiệu suất, tránh tải quá nhiều dữ liệu không cần thiết.
- `FetchType.EAGER` (Tải ngay): Dữ liệu của các Entity liên quan sẽ được tải ngay lập tức cùng với Entity chính. Đây là chế độ tải mặc định cho các mối quan hệ `@OneToOne` và `@ManyToOne`. Việc sử dụng `EAGER` loading có thể dẫn đến các vấn đề về hiệu suất (N+1 select problem) nếu không được quản lý cẩn thận, đặc biệt với các mối quan hệ `ToMany`.

3.6 Orphan Removal

`orphanRemoval = true` là một thuộc tính của các annotation mối quan hệ (`@OneToOne`, `@OneToMany`) cho phép tự động xóa các Entity con khi chúng không còn được tham chiếu bởi Entity cha. Điều này hữu ích khi vòng đời của Entity con phụ thuộc hoàn toàn vào Entity cha.

Java

```
@Entity
public class Department {
    @OneToMany(mappedBy = "department",
        cascade = CascadeType.ALL,
        orphanRemoval = true) // Khi một Employee bị gỡ bỏ khỏi danh
        sách employees của Department, Employee đó sẽ tự động bị xóa khỏi cơ sở dữ
        liệu.
    private List<Employee> employees;

    public void removeEmployee(Employee employee) {
        employees.remove(employee); // Employee sẽ bị xóa khỏi DB khi
        transaction commit
        employee.setDepartment(null); // Đảm bảo mối quan hệ ở phía Employee
        cũng được gỡ bỏ
    }
}
```

Giải thích:

- Khi `orphanRemoval = true` được đặt, nếu một Entity con bị gỡ bỏ khỏi tập hợp các Entity con của Entity cha (ví dụ: xóa khỏi danh sách `employees` của `Department`), JPA sẽ tự động xóa Entity con đó khỏi cơ sở dữ liệu khi transaction được commit.
- Điều này khác với `CascadeType.REMOVE` ở chỗ `CascadeType.REMOVE` chỉ xóa Entity con khi Entity cha bị xóa. `orphanRemoval` xử lý trường hợp Entity con bị "mồ côi" (không còn cha) ngay cả khi Entity cha vẫn tồn tại.

3.7 @JoinColumn và @JoinTable

3.7.1 @JoinColumn

Chỉ định cột foreign key. Annotation này được sử dụng để định nghĩa cột khóa ngoại trong bảng cơ sở dữ liệu, thiết lập mối quan hệ giữa hai Entity.

Java

```
@Entity
public class Employee {
    @ManyToOne // Mối quan hệ nhiều-một: nhiều Employee thuộc về một
    Department.
```

```

    @JoinColumn(name = "dept_id", // Tên của cột khóa ngoại trong bảng
Employee sẽ là dept_id.
                nullable = false, // Cột này không được phép null, nghĩa là
mỗi Employee phải thuộc về một Department.
                foreignKey = @ForeignKey(name = "FK_EMPLOYEE_DEPARTMENT")) //
Định nghĩa tên của ràng buộc khóa ngoại trong cơ sở dữ liệu.
    private Department department;
}

```

Giải thích:

- `@JoinColumn` thường được sử dụng với các mối quan hệ `@OneToOne` và `@ManyToOne` để chỉ định cột khóa ngoại trong bảng của Entity hiện tại (`Employee`) sẽ tham chiếu đến khóa chính của Entity được liên kết (`Department`).
- `name` : Tên của cột khóa ngoại trong bảng.
- `nullable` : Nếu `false` , cột khóa ngoại không thể chứa giá trị `NULL` , đảm bảo rằng mỗi `Employee` phải có một `Department` .
- `foreignKey` : Cho phép bạn đặt tên cho ràng buộc khóa ngoại trong cơ sở dữ liệu, giúp dễ dàng quản lý và tham chiếu trong các công cụ quản lý cơ sở dữ liệu.

3.7.2 @JoinTable

Chỉ định bảng trung gian cho Many-to-Many. Annotation này được sử dụng để tùy chỉnh bảng trung gian và các cột khóa ngoại của nó trong mối quan hệ nhiều-nhiều.

Java

```

@Entity
public class Student {
    @ManyToMany
    @JoinTable( // Định nghĩa bảng trung gian cho mối quan hệ nhiều-nhiều.
        name = "student_course", // Tên của bảng trung gian sẽ là
student_course.
        joinColumns = @JoinColumn(name = "student_id"), // Cột khóa ngoại
trong bảng trung gian tham chiếu đến khóa chính của Student.
        inverseJoinColumns = @JoinColumn(name = "course_id"), // Cột khóa
ngoại trong bảng trung gian tham chiếu đến khóa chính của Course.
        uniqueConstraints = @UniqueConstraint(columnNames = {"student_id",
"course_id"})) // Đảm bảo sự kết hợp của student_id và course_id là duy nhất,
ngăn chặn việc một sinh viên đăng ký cùng một khóa học nhiều lần.
}

```

```

    )
    private Set<Course> courses;
}

```

Giải thích:

- `@JoinTable` được sử dụng trong mối quan hệ `@ManyToMany` để định nghĩa bảng trung gian (join table) sẽ lưu trữ các liên kết giữa hai Entity.
- `name` : Tên của bảng trung gian trong cơ sở dữ liệu.
- `joinColumns` : Định nghĩa cột khóa ngoại trong bảng trung gian trỏ về Entity mà annotation `@JoinTable` được đặt (`Student`).
- `inverseJoinColumns` : Định nghĩa cột khóa ngoại trong bảng trung gian trỏ về Entity đối diện (`Course`).
- `uniqueConstraints` : Có thể được sử dụng để thêm các ràng buộc duy nhất trên các cột của bảng trung gian, ví dụ để đảm bảo rằng một cặp (`student_id`, `course_id`) chỉ xuất hiện một lần.

3.8 Best Practices cho Relationships

1. Sử dụng Set thay vì List cho Many-to-Many để tránh duplicate:

Java

```

private Set<Course> courses = new HashSet<>(); // Tốt
private List<Course> courses = new ArrayList<>(); // Có thể gây duplicate

```

1. Luôn đồng bộ hóa cả hai phía trong Bidirectional relationship:

Java

```

public void addEmployee(Employee employee) {
    employees.add(employee);
    employee.setDepartment(this); // Quan trọng!
}

```

1. Sử dụng LAZY loading mặc định và chỉ dùng EAGER khi thực sự cần:

Java

```
@OneToMany(fetch = FetchType.LAZY) // Mặc định, tốt  
@OneToMany(fetch = FetchType.EAGER) // Chỉ khi cần thiết
```

1. Cần thận với Cascade.REMOVE trong Many-to-Many:

Java

```
// Có thể gây xóa nhầm dữ liệu  
@ManyToMany(cascade = CascadeType.REMOVE) // Nguy hiểm!  
  
// An toàn hơn  
@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
```